COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	10/15/2026	**DATE SUBMITTED:**	10/20/2026
NAME:	Ralf Nicolo B. Abello Rodney C. Domingo	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13 **PARALLEL ALGORITHMS AND MULTITHREADING**

1. PROCEDURE OUTPUT

ILO A and B:
PART 1:

Code

HOA12 > hoa13p1 > singlethread.cpp > main()

```cpp
#include <iostream>
#include <thread>

void print(int n, const std::string &str){
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

Screenshot

```
PS C:\Users\TIPQC\Documents\P
  ($?) { g++ hoa13.cpp -o hoa1
Printing integer: 10
Printing string: T.I.P.
```

```
PS C:\Users\TIPQC\Documents\
hoa13p1\" ; if ($?) { g++ ho
Printing integer: 10
Printing string: T.I.P.
```

Analysis

In this program, we created a single thread t1 using std::thread, passing the function print along with the arguments 10 and "T.I.P.". The thread ran print, which displayed "Printing integer: 10" and "Printing string: T.I.P." as shown in the screenshots. we called t1.join() so that the main function waits for the thread to finish before reaching return 0, which prevents the program from ending while the thread is still running. Since only one thread does the work and main just waits, the output is the same as a normal sequential call to print, so this example shows the basic syntax of creating and joining a thread rather than any speedup from multithreading.

Table 13-1. Simple One-Threaded Example

Part 2:

Code	HOA12 > hoa13p1 > multi.cpp > main()
	1 #include <iostream>
	2 #include <thread>
	3 #include <vector>
	4
	5 void print(int n, const std::string &str){
	6 std::string msg = std::to_string(n) + " : " + str;
	7 std::cout << msg << std::endl;
	8 }
	9
	10 int main() {
	11 std::vector<std::string> s = {
	12 "T.I.P.",
	13 "Competent",
	14 "Computer",
	15 "Engineers"
	16 };
	17 std::vector<std::thread> threads;
	18
	19 for (int i = 0; i < s.size(); i++){
	20 threads.push_back(std::thread(print, i , s[i]));
	21 }
	22
	23 for (auto &th : threads) {
	24 th.join();
	25 }
	26 return 0;
	27 }
	28

Screenshot	0 : T.I.P.2 : Computer 1 : Competent 3 : Engineers **Abello** PS C:\Users\TIPQC\Documen hoa13p1\" ; if ($?) { g++ 0 : T.I.P.1 : Competent 2 : Computer 3 : Engineers **Domingo**
Analysis	This program is an example of a multithread code that demonstrates concurrent execution. The print function formats an integer index and a string parameter into a combined message before putting it into the console. Inside the main function, a vector of strings containing the words from the example code given is initialized. A loop spawns multiple threads assigning each one an index and a corresponding string from the vector. Finally, another loop calls the "th.join()" function on each thread to ensure the main program waits for all concurrent tasks to finish.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PART A:

1. Write a definition of multithreading and its advantages/disadvantages.

Abello	I define multithreading as a technique where a single process runs multiple threads that share the same memory space. Which I can do in c++ with the std::thread class. The advantages I see are better CPU utilization and a more responsive program, while the disadvantages are added complexity, race conditions and deadlocks, and harder debugging.
Domingo	Multithreading is a type of programming and execution models where one process is divided into multiple concurrent execution threads that share the same memory space and resources while executing individually. One of the advantages of multithreading is responsiveness to allow apps to remain interactive even if a part of the program is doing a long task. Another advantage of this is the ability to share resources, it shares the memory and resources of the parent process directly, making the communication and data sharing of both applications more efficient. While the disadvantages of this is its complexity and the debugging part of the program, this introduces bugs like race conditions and deadlocks which reproduce.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Abello	First, in a game, I would use one thread for rendering and another for user input so the program never freezes. Second, in a web server.I would give each client request it own thread, and third, in data processing.I would split a large datasheet among threads to cut computation time on a multi-core processor
Domingo	A key use-case for multithreading is web server management, where an incoming client request is handled on a dedicated thread so that thousands of user connections can be processed concurrently. A second use-case is found within GUIs or Graphical User Interfaces, where background threads perform heavy operations like file downloads while the main thread stays interactive. A third and final example is gaming engines, where separate threads independently process physics calculations, AI, audio playback, and graphics rendering.

3. Differentiate between parallelism and concurrency.

Abello	I understand concurrency as managing multiple tasks over overlapping time periods by taking turns, even on a single CPU core. Parallelism means actually executing multiple tasks at the same instant, which requires multiple cores.
Domingo	Both parallelism and concurrency describe handling multiple paths, they represent fundamentally district concepts. Concurrency is all about application structure and refers to a system's ability to deal with multiple tasks by interleaving their execution over overlapping time periods. A single-core CPU can actually achieve concurrency by switching rapidly between different tasks, creating the illusion that they are running continuously. Parallelism is all about hardware execution and requires physically running

	multiple tasks at the exact same time. Parallelism cannot occur on a single-core system because it strictly depends on multi-core processors, multi-CPU systems or distributed clusters.

PART B:

CODE (Abello)

```cpp
#include <iostream>
#include <thread>

int globalVar = 0;

void add(int value) {
    globalVar += value;
}

int main() {
    std::thread t1(add, 10);
    std::thread t2(add, 20);
    std::thread t3(add, 30);

    std::cout << "Display (before any join): " << globalVar << std::endl;
    t1.join();
    std::cout << "After T1.join():           " << globalVar << std::endl;
    t2.join();
    std::cout << "After T2.join():           " << globalVar << std::endl;
    t3.join();
    std::cout << "After T3.join():           " << globalVar << std::endl;
    return 0;
}
```

```
Display (before any join): 0
After T1.join():           60
After T2.join():           60
After T3.join():           60
```

CODE (Domingo)

```cpp
#include <iostream>
#include <thread>
#include <chrono>

int global_counter = 0;

void add_value(int val) {
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
    global_counter += val;
}

int main() {
    global_counter = 0;
    std::cout << "Initial global_counter: " << global_counter << std::endl;

    std::thread t1(add_value, 10);
    std::thread t2(add_value, 20);
    std::thread t3(add_value, 30);

    std::cout << "[Before T1.join()] Current global_counter: " << global_counter << std::endl;
    t1.join();
    std::cout << "[After T1.join()]  Current global_counter: " << global_counter << std::endl;
    std::cout << "[Before T2.join()] Current global_counter: " << global_counter << std::endl;
    t2.join();
    std::cout << "[After T2.join()]  Current global_counter: " << global_counter << std::endl;
    std::cout << "[Before T3.join()] Current global_counter: " << global_counter << std::endl;
    t3.join();
    std::cout << "[After T3.join()]  Final global_counter: " << global_counter << std::endl;

    return 0;
}
```

```
[Before T1.join()] Current global_counter: 0
[After T1.join()]  Current global_counter: 30
[Before T2.join()] Current global_counter: 30
[After T2.join()]  Current global_counter: 30
[Before T3.join()] Current global_counter: 30
[After T3.join()]  Final global_counter: 30
```

ANALYSIS	
Abello	In this exercise, I created a global integer and three threads that each called the **add** function with the values 10, 20, and 30. The first value I displayed, before any **join()**, was 0 because the main thread printed the variable before the other threads had finished adding to it. After calling **T1.join()**, the main thread waited only for T1 to finish, but T2 and T3 were running at the same time, so by then the value had already reached 60 in my run. This showed me that **join()** only guarantees that the specific thread being joined has finished, so the values displayed between joins can change from run to run. The final value after all

	three joins is the only one I can rely on. I also noticed that all three threads modify the same global variable without any protection such as a mutex, so there may be a data race. If two threads read **globalVar** at the same time, one update could be lost and the result would be wrong, or the behavior could even be undefined. The program printed the correct total of 60, but that does not mean it is always safe, since the race only shows up occasionally. This taught me that sharing a variable between threads needs synchronization to be reliable.
Domingo	In Part B, I created three threads that executed **add_value** to add 10, 20, and 30 to **global_counter** after a 100ms sleep delay. Before calling any **join()**, **global_counter** was 0 because the main thread printed its value before the threads finished sleeping and updating the variable. After waiting for **t1.join()**, the value jumped straight to 30. This occurred because all three threads slept for the same 100ms duration and completed their updates nearly simultaneously around the time **t1.join()** finished. Consequently, subsequent checks before and after joining **t2** and **t3** remained at 30. However, because multiple threads modified the global variable without synchronization like a mutex, a race condition occurred where concurrent writes overlapped and updates were overwritten. This exercise highlights that thread joining only controls execution waiting, whereas shared data access requires proper synchronization mechanisms.

PART C:

CODE (Abello)

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>

void merge(std::vector<int>& a, int l, int m, int r) {
    std::vector<int> tmp;
    int i = l, j = m + 1;
    while (i <= m && j <= r) {
        if (a[i] <= a[j]) tmp.push_back(a[i++]);
        else tmp.push_back(a[j++]);
    }
    while (i <= m) tmp.push_back(a[i++]);
    while (j <= r) tmp.push_back(a[j++]);
    for (int k = 0; k < tmp.size(); k++) a[l + k] = tmp[k];
}

void mergeSort(std::vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}

int main() {
    int n = 5000000;
    std::vector<int> a(n);
    for (int i = 0; i < n; i++) a[i] = (i * 7919LL) % 1000003;
    std::vector<int> b = a;
    int mid = (n - 1) / 2;

    // Normal merge sort (one thread)
    auto start1 = std::chrono::steady_clock::now();
    mergeSort(a, 0, n - 1);
    auto end1 = std::chrono::steady_clock::now();

    // Multithreaded: two threads sort each half, then merge
    auto start2 = std::chrono::steady_clock::now();
    std::thread t1([&b, mid] { mergeSort(b, 0, mid); });
    std::thread t2([&b, mid, n] { mergeSort(b, mid + 1, n - 1); });
    t1.join();
    t2.join();
    merge(b, 0, mid, n - 1);
    auto end2 = std::chrono::steady_clock::now();

    std::cout << "Single thread: "
              << std::chrono::duration<double, std::milli>(end1 - start1).count() << " ms" << std::endl;
    std::cout << "Two threads:   "
              << std::chrono::duration<double, std::milli>(end2 - start2).count() << " ms" << std::endl;
    std::cout << "Same result: " << (a == b) << std::endl;
    return 0;
}
```

```
Single thread: 48514.1 ms
Two threads:   62705 ms
Same result: 1
```

CODE (Domingo)

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <numeric>
#include <chrono>

void partial_sum(const std::vector<long long>& data, size_t start, size_t end, long long& result) {
    long long local_sum = 0;
    for (size_t i = start; i < end; ++i) {
        local_sum += data[i];
    }
    result = local_sum;
}

int main() {

    const size_t ARRAY_SIZE = 100'000'000;
    std::vector<long long> numbers(ARRAY_SIZE, 1);

    auto start_seq = std::chrono::high_resolution_clock::now();
    long long seq_sum = 0;
    for (size_t i = 0; i < ARRAY_SIZE; ++i) {
        seq_sum += numbers[i];
    }
    auto end_seq = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> seq_duration = end_seq - start_seq;

    std::cout << "Sequential Sum Result: " << seq_sum << std::endl;
    std::cout << "Sequential Time: " << seq_duration.count() << " ms\n" << std::endl;

    unsigned int num_threads = std::thread::hardware_concurrency();
    if (num_threads == 0) num_threads = 4; // Default fallback

    std::vector<std::thread> threads;
    std::vector<long long> partial_results(num_threads, 0);
    size_t chunk_size = ARRAY_SIZE / num_threads;

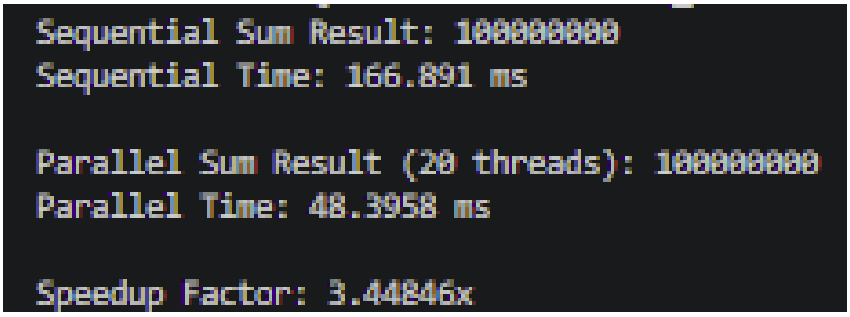
    auto start_par = std::chrono::high_resolution_clock::now();

    for (unsigned int i = 0; i < num_threads; ++i) {
        size_t start_idx = i * chunk_size;
        size_t end_idx = (i == num_threads - 1) ? ARRAY_SIZE : start_idx + chunk_size;
        threads.push_back(std::thread(partial_sum, std::cref(numbers), start_idx, end_idx, std::ref(partial_results[i])));
    }
    for (auto& th : threads) {
        th.join();
    }
    long long parallel_sum = 0;
    for (long long res : partial_results) {
        parallel_sum += res;
    }

    auto end_par = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> par_duration = end_par - start_par;

    std::cout << "Parallel Sum Result (" << num_threads << " threads): " << parallel_sum << std::endl;
    std::cout << "Parallel Time: " << par_duration.count() << " ms\n" << std::endl;
    std::cout << "Speedup Factor: " << seq_duration.count() / par_duration.count() << "x" << std::endl;

    return 0;
}
```



ANALYSIS	
Abello	In Part C, I used merge sort on an array of 5,000,000 integers and compared a normal single-threaded run against a version that used two threads to sort each half of the array. After both threads finished, I merged the halves with one final **merge** call. The single-threaded sort took 48514.1 ms, while the two-thread version took 62705 ms, so my multithreaded version was actually slower, not faster, which was not what I expected. The output **Same result: 1** showed that both versions sorted the array correctly, so the slowdown was only about speed, not accuracy. There was no data race here because **t1** sorted only the first half and **t2** sorted only the second half, so the threads never touched the same elements. I think the slowdown happened because the work in my **merge** function is heavy on memory, since it creates a temporary vector with **push_back** for every merge, so both threads were competing for memory access and allocation at the same time. The final **merge** also ran on a single thread, which means part of the work could never be parallelized, and the cost of creating the threads and calling **join()** added overhead too. This taught me that using more threads does not always make a program faster, and that the speedup depends on the hardware, how the work is split, and how much of the algorithm has to stay sequential.
Domingo	In Part C, I implemented a parallel reduction algorithm in C++ to calculate the sum of 100,000,000 elements. The sequential approach completed in 166.891 ms, whereas distributing the workload across 20 worker threads using **std::thread::hardware_concurrency** reduced the execution time to 48.3958 ms. This achieved a speedup factor of 3.44846x. To prevent race conditions and lock contention on a shared total, each thread processed a distinct memory chunk and stored its subtotal in a pre-allocated vector before performing a final sequential reduction in **main()**. This demonstrates that dividing data into independent chunks and accumulating intermediate results effectively leverages multi-core processors for parallel speedup without synchronization overhead.

CONCLUSION	
Abello	In this Hands-On Activity, I learned how to create and manage threads in C++ using **std::thread** and how **join()** makes the main function wait for a thread to finish. In the

	procedure, the one-thread example only showed the basic syntax, while the multithreaded example printed its lines out of order and mixed together (for example, line 2 appeared before lines 1 and 3 in my output), which showed me that threads run at the same time and that their order is not guaranteed. In Part B, the value of **globalVar** was 0 before any **join()** and already 60 after **T1.join()**, so I learned that **join()** only guarantees the thread being joined has finished, and that sharing one variable between threads without a mutex can cause a data race. In Part C, my two-thread merge sort took 62705 ms compared to 48514.1 ms for the single-thread version, so it was slower, which taught me that more threads do not always mean better performance because of overhead, memory use, and the part of the algorithm that stays sequential. Overall, I think I did well in understanding how the code works and explaining the results, but I need to improve in using synchronization tools like **std::mutex** and in writing code that actually gets a speedup from multithreading.
Domingo	In this activity, I explored the core concepts of multithreading, concurrency, and parallelism in C++. In Part A, I defined multithreading as dividing a process into concurrent execution threads that share memory, which enhances responsiveness and resource sharing but introduces synchronization challenges like race conditions. I also clarified that concurrency manages multiple tasks by interleaving execution, whereas parallelism physically runs tasks simultaneously across multiple CPU cores. In Part B, my analysis demonstrated the behavior of thread joining and shared memory access; because all threads slept for 100ms, the counter value jumped to 30 upon joining the first thread, illustrating how un-synchronized concurrent writes to shared data cause race conditions. Finally, in Part C, I successfully implemented a parallel reduction algorithm using 20 worker threads via **std::thread::hardware_concurrency**, reducing the execution time from 166.891 ms to 48.3958 ms for a 3.45x speedup. By dividing data into independent chunks and storing intermediate subtotals, race conditions were avoided without lock contention. Overall, this activity provided valuable hands-on experience in effectively structuring multithreaded applications and leveraging multi-core hardware for performance optimization.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.